RESEARCH PAPER

RIGOR

Retrieval-Augmented Generation of Ontologies from Relational Databases

Nadeen Fathallah[1] · Mojtaba Nayyeri[1] · Athish A Yogi[1]

Ratan Bahadur Thapa[1] · Hans-Michael Tautenhahn[3]

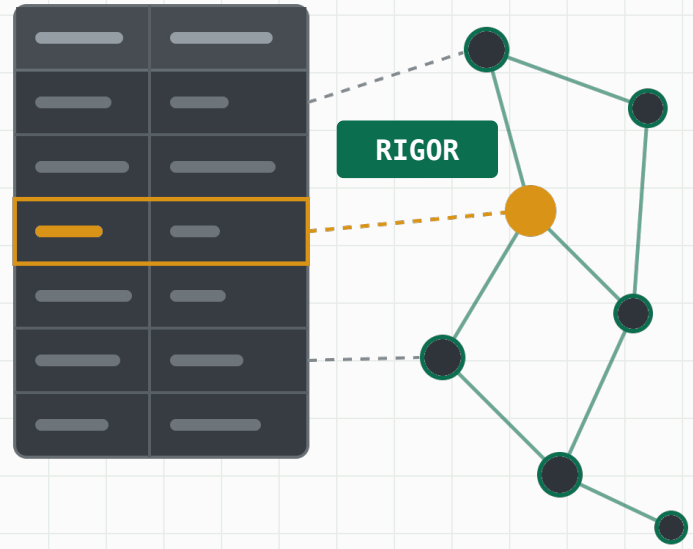
Anton Schnurpel[3] · Steffen Staab[1,2]

[1] Institute for Artificial Intelligence, University of Stuttgart, Germany

[2] University of Southampton, United Kingdom

[3] Universitätsklinikum Leipzig, Germany

PAPER arXiv:2506.01232 CODE github.com/NadeenAhmad/RIGOR

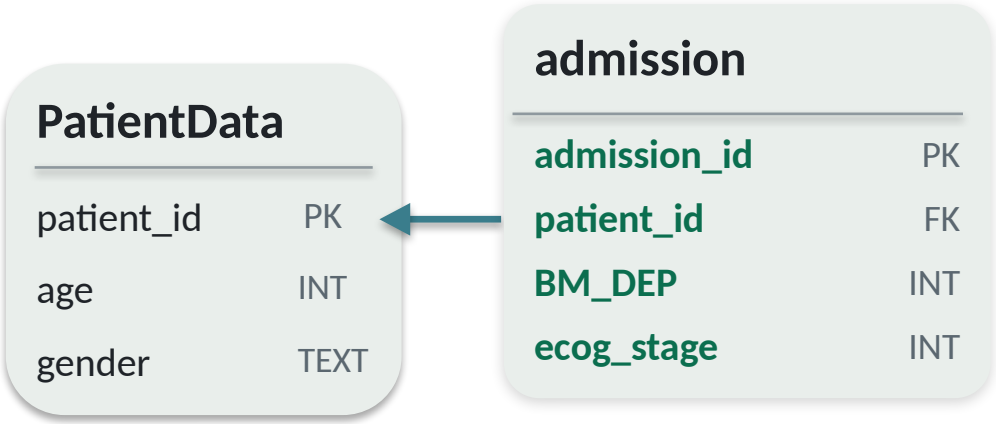


ISWC 2026

Bari, Italy

A database can tell us how data is stored. Can it tell us what the data means?

RELATIONAL SCHEMA · EXCERPT



The FK links an **admission** to a **patient**.

WHAT WE STILL NEED TO UNDERSTAND

01 `patient_id` — How do we represent the **link** from an **admission** to its **patient**?

02 `BM_DEP` — Which **clinical concept** does this **abbreviation mean**?

03 `ecog_stage` — What do the **encoded** score **values mean**?

Those meanings matter for ontology-based access, knowledge graphs and integration.

WHAT WE WANT TO BUILD

Not just a cleaner schema.
A usable semantic model.

INPUT

admission

admission_id	PK
patient_id	FK
BM_DEP	INT
ecog_stage	INT



TARGET

WHAT WE WANT TO BUILD

Not just a cleaner schema. A usable semantic model.

INPUT

admission

admission_id	PK
patient_id	FK
BM_DEP	INT
ecog_stage	INT



TARGET

Admission —hasPatientData→ PatientData

```
Class: Admission
  SubClassOf:
    admission_id some xsd:integer,
    hasPatientData some PatientData,        ← schema (foreign key)
    BM_DEP only xsd:integer,
    ecog_stage only xsd:integer

DataProperty: admission_id
ObjectProperty: hasPatientData              ← schema (foreign key)
DataProperty: BM_DEP
  Annotations:


    prov:wasDerivedFrom admission:BM_DEP    ← source column
DataProperty: ecog_stage
  Annotations:


    prov:wasDerivedFrom admission:ecog_stage ← source column
```

WHAT WE WANT TO BUILD

Not just a cleaner schema. A usable semantic model.

INPUT

admission

admission_id	PK
patient_id	FK
BM_DEP	INT
ecog_stage	INT



TARGET

Admission —hasPatientData→ PatientData

```
Class: Admission
  SubClassOf:
    admission_id some xsd:integer,
    hasPatientData some PatientData,          ← schema (foreign key)
    BM_DEP only xsd:integer,
    ecog_stage only xsd:integer

DataProperty: admission_id
ObjectProperty: hasPatientData                ← schema (foreign key)
DataProperty: BM_DEP
  Annotations:
    rdfs:label "Bone Marrow Depression"@en,   ← documentation

    prov:wasDerivedFrom admission:BM_DEP      ← source column
DataProperty: ecog_stage
  Annotations:
    rdfs:label "ECOG Performance Status"@en,  ← documentation

    prov:wasDerivedFrom admission:ecog_stage  ← source column
```

WHAT WE WANT TO BUILD

Not just a cleaner schema. A usable semantic model.

INPUT

admission

admission_id	PK
patient_id	FK
BM_DEP	INT
ecog_stage	INT



TARGET

Admission —hasPatientData→ PatientData

```
Class: Admission
  SubClassOf:
    admission_id some xsd:integer,
    hasPatientData some PatientData,          ← schema (foreign key)
    BM_DEP only xsd:integer,
    ecog_stage only xsd:integer

DataProperty: admission_id
ObjectProperty: hasPatientData                ← schema (foreign key)
DataProperty: BM_DEP
  Annotations:
    rdfs:label "Bone Marrow Depression"@en,   ← documentation
    skos:relatedMatch obo:DOID_4961,          ← Disease Ontology
    prov:wasDerivedFrom admission:BM_DEP      ← source column
DataProperty: ecog_stage
  Annotations:
    rdfs:label "ECOG Performance Status"@en,  ← documentation

    prov:wasDerivedFrom admission:ecog_stage  ← source column
```

WHAT WE WANT TO BUILD

Not just a cleaner schema. A usable semantic model.

INPUT

admission

admission_id	PK
patient_id	FK
BM_DEP	INT
ecog_stage	INT



TARGET

Admission —hasPatientData→ PatientData

```
Class: Admission
  SubClassOf:
    admission_id some xsd:integer,
    hasPatientData some PatientData,          ← schema (foreign key)
    BM_DEP only xsd:integer,
    ecog_stage only xsd:integer

DataProperty: admission_id
ObjectProperty: hasPatientData                ← schema (foreign key)
DataProperty: BM_DEP
  Annotations:
    rdfs:label "Bone Marrow Depression"@en,   ← documentation
    skos:relatedMatch obo:DOID_4961,          ← Disease Ontology
    prov:wasDerivedFrom admission:BM_DEP      ← source column
DataProperty: ecog_stage
  Annotations:
    rdfs:label "ECOG Performance Status"@en,  ← documentation
    rdfs:comment "0 = fully active … 5 = dead"@en,
    prov:wasDerivedFrom admission:ecog_stage  ← source column
```

WHAT WE WANT TO BUILD

Not just a cleaner schema. A usable semantic model.

INPUT

admission

admission_id	PK
patient_id	FK
BM_DEP	INT
ecog_stage	INT

WHAT THIS ENABLES

Ontology-based data access (OBDA)
Knowledge graph population
Semantic data integration

TARGET

Admission —hasPatientData→ PatientData

```
Class: Admission
  SubClassOf:
    admission_id some xsd:integer,
    hasPatientData some PatientData,        ← schema (foreign key)
    BM_DEP only xsd:integer,
    ecog_stage only xsd:integer

DataProperty: admission_id
ObjectProperty: hasPatientData              ← schema (foreign key)
DataProperty: BM_DEP
  Annotations:
    rdfs:label "Bone Marrow Depression"@en,        ← documentation
    skos:relatedMatch obo:DOID_4961,               ← Disease Ontology
    prov:wasDerivedFrom admission:BM_DEP           ← source column
DataProperty: ecog_stage
  Annotations:
    rdfs:label "ECOG Performance Status"@en,       ← documentation
    rdfs:comment "0 = fully active … 5 = dead"@en,
    prov:wasDerivedFrom admission:ecog_stage       ← source column
```

THE RESEARCH GAP

Two partial solutions. Neither solves the whole problem.

SCHEMA-FIRST RULES

Preserve the structure

Tables, columns and foreign keys can be mapped systematically.

But implicit **domain semantics** remain **missing**.

W3C Direct Mapping [Arenas et al. 2012; Sequeda et al. 2012] · **BootOX** [Jiménez-Ruiz et al., ISWC 2015] · **Ben Mahria et al.** [J. Big Data 2021]

LLM-FIRST GENERATION

Suggest the meaning

Labels, concepts and plausible relationships can be proposed.

But elements may be **omitted** or modeled incorrectly.

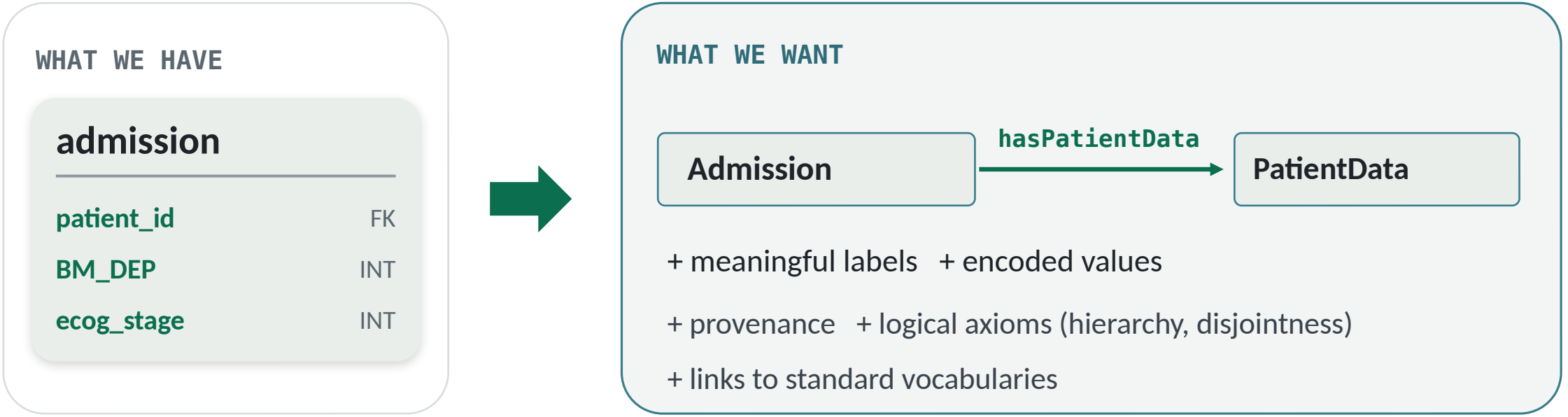
LLMs4OL [Babaei Giglou et al., ISWC 2023] · **Mateiu & Groza** [SYNASC 2023] · **OntoGenix** [Val-Calvo et al., IP&M 2025] · **LLM4VKG** [Xiao et al., IJCAI 2025]

OUR QUESTION **Can we preserve schema fidelity and add domain semantics across the full database?**

OUR ANSWER / RIGOR

Build the ontology one table at a time.

We will construct the answer to our admission example, together.



We need to preserve structure, discover meaning, and prevent mistakes, at database scale.

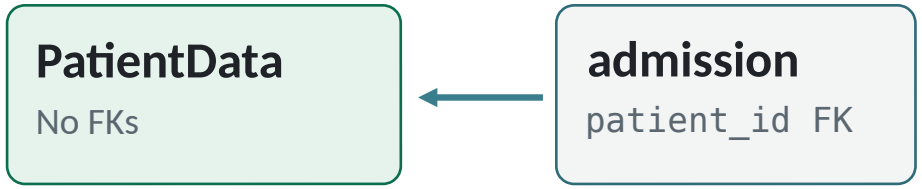
1 ORDER | 2 SEED | 3 RETRIEVE | 4 GENERATE | 5 VALIDATE | 6 MERGE

STEP 1 / ORDER

To understand this table, build its parent first.

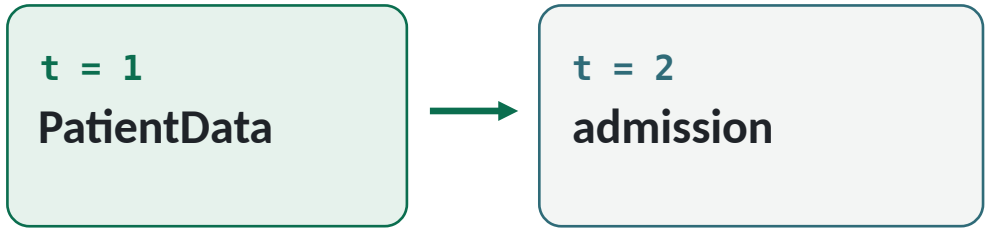
We follow foreign-key dependencies, so known concepts are available when needed.

DATABASE DEPENDENCIES



FK points to parent

PROCESSING ORDER / OUTPUT



Before generating `admission`, `PatientData` already exists in the core ontology.

WHY IT MATTERS The next fragment can **reuse** known **classes** and **IRIs** instead of inventing duplicates.

STEP 2 / SEED

Guarantee schema coverage before LLM enrichment.

Direct Mapping (DM_r) represents every column in the table before generation.

INPUT

admission

patient_id	FK → PatientData
BM_DEP	INT
ecog_stage	INT

OUTPUT / DM_r

```
Class: Admission
  SubClassOf:
    admission_id some xsd:integer,
    patient_id some PatientData,        ← each has a patient
    BM_DEP only xsd:integer,            ← if present, an integer
    ecog_stage only xsd:integer
DataProperty: admission_id
ObjectProperty: patient_id              ← the FK becomes a link
DataProperty: BM_DEP
DataProperty: ecog_stage
```

GUARANTEE Every schema column is represented in the deterministic seed.

1 ORDER | 2 SEED | 3 RETRIEVE | 4 GENERATE | 5 VALIDATE | 6 MERGE

STEP 3 / RETRIEVE

The draft is structurally faithful. But what does it mean?

RIGOR gathers a **context** package from **three** complementary **sources**.

1 DOCUMENTATION



What do column names mean?

`BM_DEP` → Bone Marrow Depression

Defines cryptic names

STEP 3 / RETRIEVE

The draft is structurally faithful. But what does it mean?

RIGOR gathers a **context** package from **three** complementary **sources**.

1 DOCUMENTATION

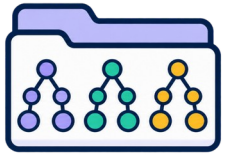


What do column names mean?

BM_DEP → Bone Marrow Depression

Defines cryptic names

2 EXTERNAL ONTOLOGIES



Which domain concepts exist?

Disease Ontology
Cell Ontology

Suggests established concepts

STEP 3 / RETRIEVE

The draft is structurally faithful. But what does it mean?

RIGOR gathers a **context** package from **three** complementary **sources**.

1 DOCUMENTATION

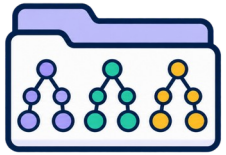


What do column names mean?

BM_DEP → Bone Marrow Depression

Defines cryptic names

2 EXTERNAL ONTOLOGIES

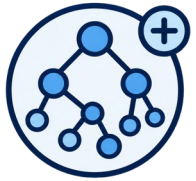


Which domain concepts exist?

Disease Ontology
Cell Ontology

Suggests established concepts

3 GROWING CORE



What have we already defined?

PatientData + previously accepted ontology terms

Reuses established concepts

1 ORDER | 2 SEED | **3 RETRIEVE** | 4 GENERATE | 5 VALIDATE | 6 MERGE

STEP 3 / RETRIEVE

The draft is structurally faithful. But what does it mean?

RIGOR gathers a **context** package from **three** complementary **sources**.

1 DOCUMENTATION

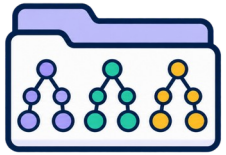


What do column names mean?

BM_DEP → Bone Marrow Depression

Defines cryptic names

2 EXTERNAL ONTOLOGIES

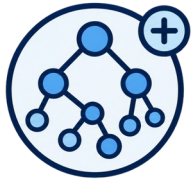


Which domain concepts exist?

Disease Ontology
Cell Ontology

Suggests established concepts

3 GROWING CORE



What have we already defined?

PatientData + previously accepted ontology terms

Reuses established concepts

ONE RETRIEVAL QUERY PER TABLE **table** + **columns** + **FK targets** → top-K per **source**

(e.g., ***admission → query: admission + patient_id, BM_DEP, ecog_stage + PatientData****)*

1 ORDER | 2 SEED | **3 RETRIEVE** | 4 GENERATE | 5 VALIDATE | 6 MERGE

STEP 4 / GENERATE

The LLM enriches a schema-grounded draft.

We prompt the ***Gen-LLM*** with the **direct mapping**, the **retrieved context**, and **semantic enrichment instructions**.

BEFORE / DIRECT MAPPING

```
Class: Admission
  SubClassOf:
    admission_id some xsd:integer,
    patient_id some PatientData,
    BM_DEP only xsd:integer,
    ecog_stage only xsd:integer

DataProperty: admission_id
ObjectProperty: patient_id
DataProperty: BM_DEP



DataProperty: ecog_stage
```

```
Class: Admission
  SubClassOf:
    admission_id some xsd:integer,
    patient_id some PatientData,
    BM_DEP only xsd:integer,
    ecog_stage only xsd:integer

DataProperty: admission_id
ObjectProperty: patient_id
DataProperty: BM_DEP



DataProperty: ecog_stage
```

1 ORDER | 2 SEED | 3 RETRIEVE | **4 GENERATE** | 5 VALIDATE | 6 MERGE

STEP 4 / GENERATE

The LLM enriches a schema-grounded draft.

We prompt the ***Gen-LLM*** with the **direct mapping**, the **retrieved context**, and **semantic enrichment instructions**.

BEFORE / DIRECT MAPPING

```
Class: Admission
  SubClassOf:
    admission_id some xsd:integer,
    patient_id some PatientData,
    BM_DEP only xsd:integer,
    ecog_stage only xsd:integer

DataProperty: admission_id
ObjectProperty: patient_id
DataProperty: BM_DEP



DataProperty: ecog_stage
```

Gen-LLM



AFTER / ENRICHED ΔO_r (delta ontology)

```
Class: Admission
  SubClassOf:
    admission_id some xsd:integer,
    hasPatientData some PatientData,                    SCHEMA
    BM_DEP only xsd:integer,
    ecog_stage only xsd:integer

DataProperty: admission_id
ObjectProperty: hasPatientData                          SCHEMA
DataProperty: BM_DEP
  Annotations:
    rdfs:label "Bone Marrow Depression"@en,             DOCS
    skos:relatedMatch obo:DOID_4961,                    ONTOLOGY
    prov:wasDerivedFrom admission:BM_DEP                SCHEMA
DataProperty: ecog_stage
  Annotations:
    rdfs:label "ECOG Performance Status"@en,            DOCS
    rdfs:comment "0 = fully active … 5 = dead"@en,      LLM
    prov:wasDerivedFrom admission:ecog_stage            SCHEMA
```

1 ORDER | 2 SEED | 3 RETRIEVE | **4 GENERATE** | 5 VALIDATE | 6 MERGE

STEP 5 / VALIDATE

Why validate? Plausible fragments can still break the ontology.

STRUCTURAL ERROR · GRAPH CHECKS



Later table **lab_result** **reuses** a core property:

```
ObjectProperty: hasPatientData  Domain: LabResult
```

Merged with **hasPatientData Domain: Admission**, OWL infers that every lab result with patient data is an admission (pitfall P19). Graph checks allow **one domain per property**.

STEP 5 / VALIDATE

Why validate? Plausible fragments can still break the ontology.

STRUCTURAL ERROR · GRAPH CHECKS



Later table **lab_result** reuses a core property:

```
ObjectProperty: hasPatientData  Domain: LabResult
```

Merged with **hasPatientData Domain: Admission**, OWL infers that every lab result with patient data is an admission (pitfall P19). Graph checks allow **one domain per property**.

SEMANTIC ERROR · JUDGE-LLM



Generated link for **admission**:

```
ObjectProperty: hasPatient  Range: Patient
```

Valid OWL, but the **core ontology** already has **PatientData**; now **two classes** mean the **same thing**. The **JUDGE-LLM** sees the core and asks to **reuse** it.

STEP 5 / VALIDATE

Why validate? Plausible fragments can still break the ontology.

STRUCTURAL ERROR · GRAPH CHECKS



Later table **lab_result** reuses a core property:

```
ObjectProperty: hasPatientData  Domain: LabResult
```

Merged with **hasPatientData Domain: Admission**, OWL infers that every lab result with patient data is an admission (pitfall P19). Graph checks allow **one domain per property**.

SEMANTIC ERROR · JUDGE-LLM



Generated link for **admission**:

```
ObjectProperty: hasPatient  Range: Patient
```

Valid OWL, but the **core ontology** already has **PatientData**; now **two classes** mean the **same thing**. The **JUDGE-LLM** sees the core and asks to **reuse** it.

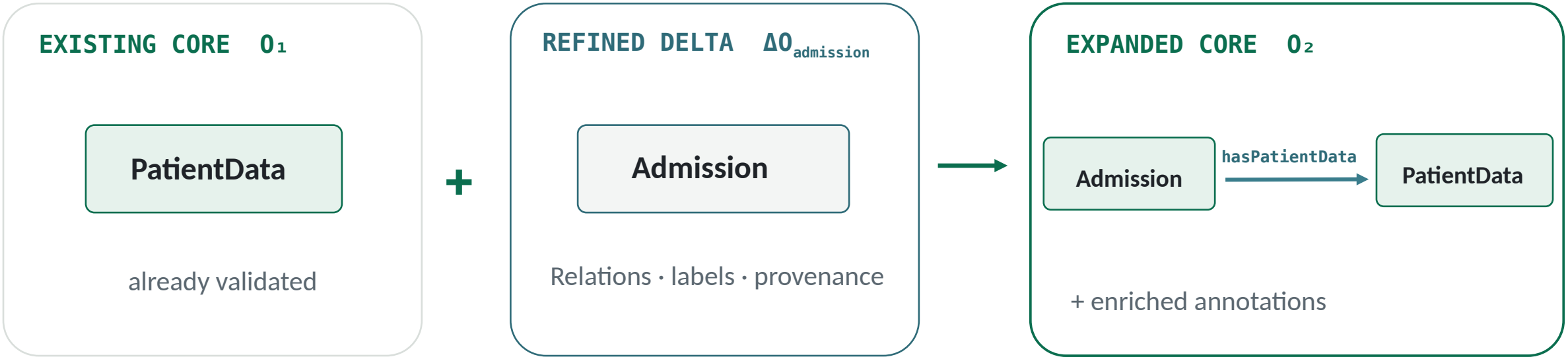
Problems go back to the ***Gen-LLM*** (up to 2 retries). If it **still fails**, the ***Judge-LLM*** picks the better of the ***Gen-LLM***'s final version and the plain ***direct mapping***.

1 ORDER | 2 SEED | 3 RETRIEVE | 4 GENERATE | **5 VALIDATE** | 6 MERGE

STEP 6 / MERGE

The ontology we built now helps build the next part.

Integrate the refined fragment; use the **expanded** core as retrieval context.

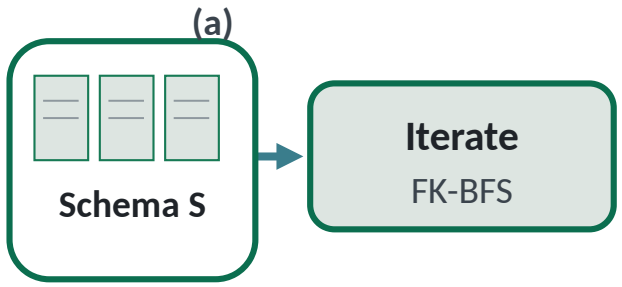


THE LOOP O_2 becomes a retrieval source for the next table → a coherent ontology **grows**.

1 ORDER | 2 SEED | 3 RETRIEVE | 4 GENERATE | 5 VALIDATE | 6 MERGE

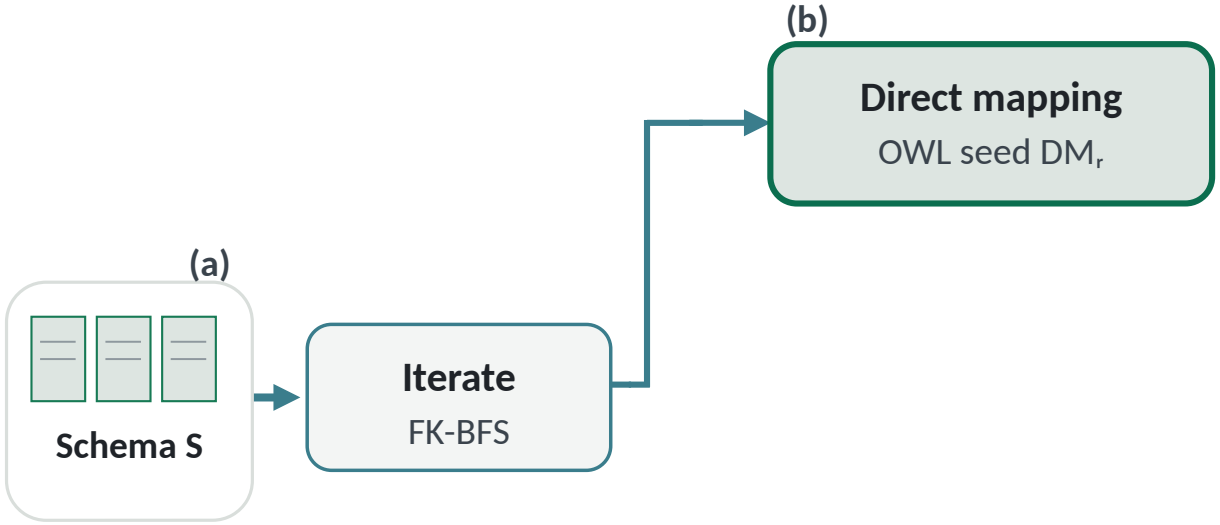
1. Follow the foreign keys

Start where the foreign-key dependencies begin.



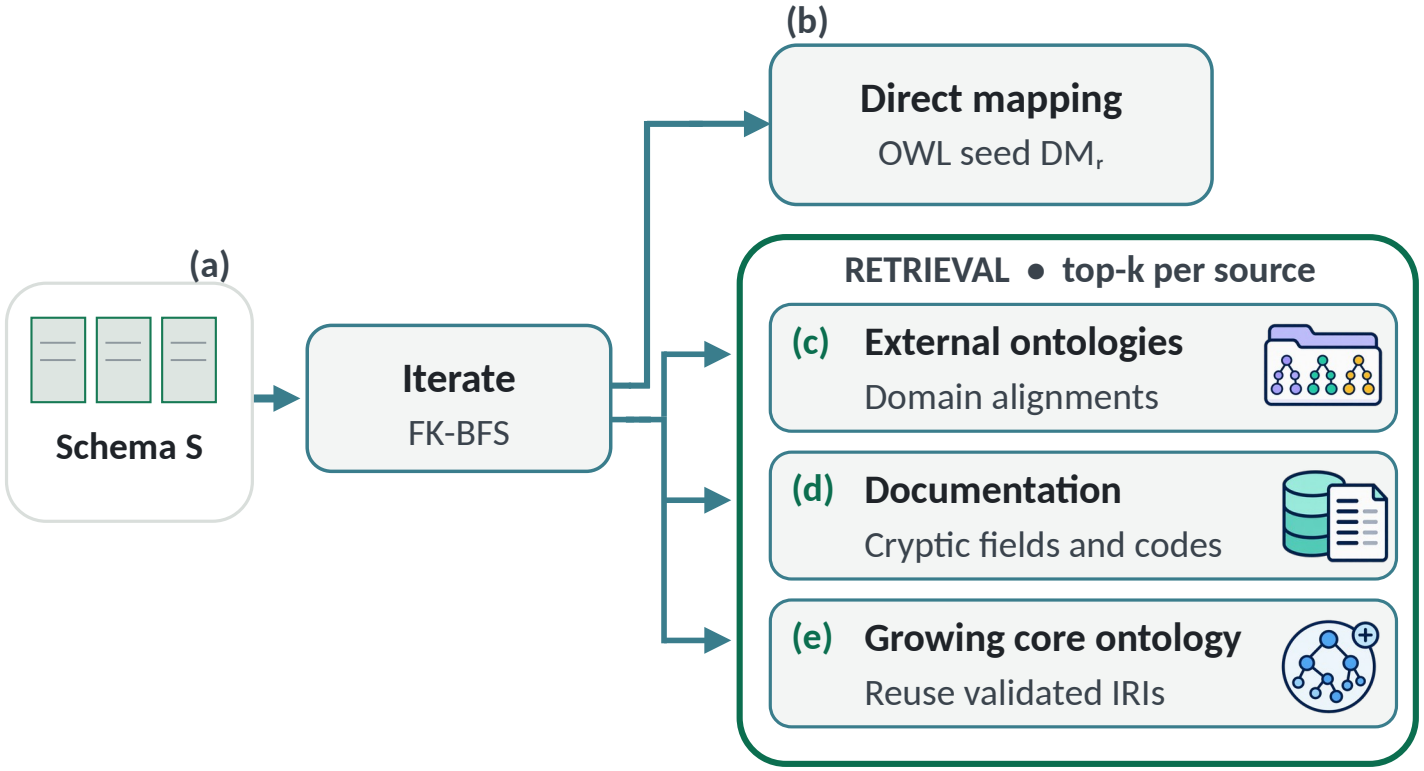
2. Anchor generation in the schema

The schema-grounded branch feeds the generation step.



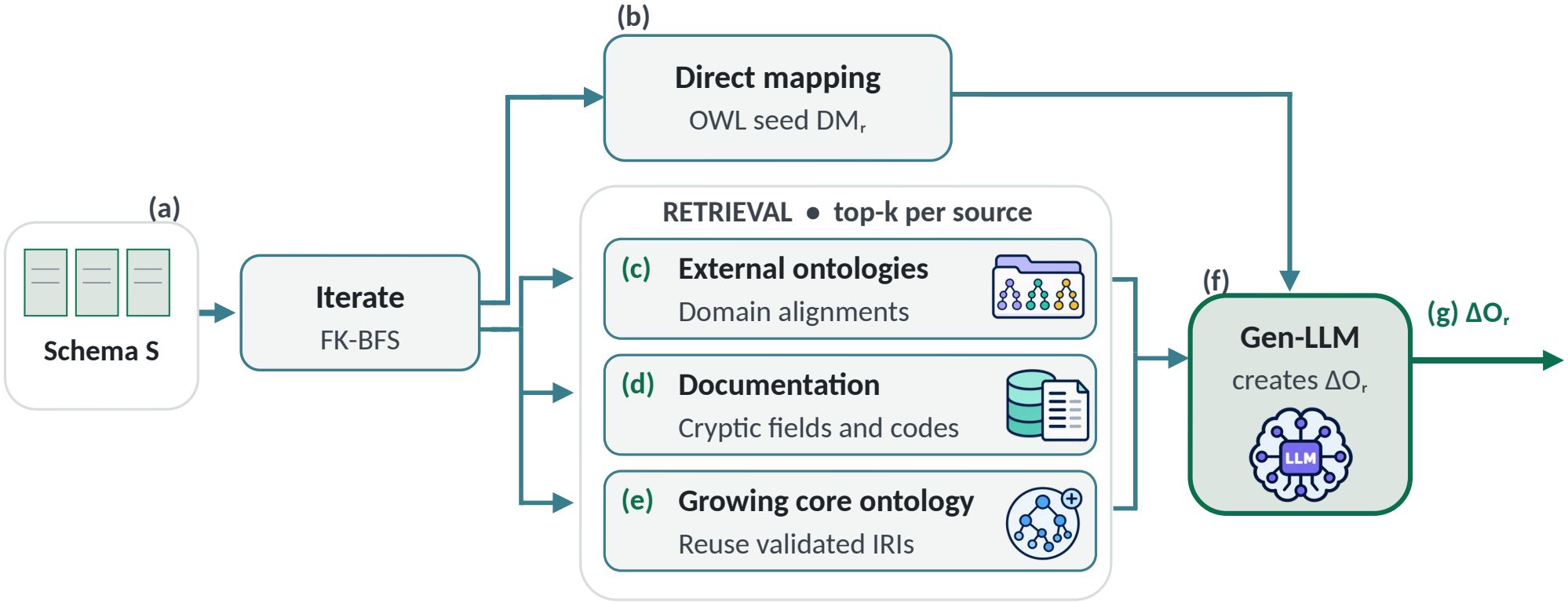
3. Retrieve the missing semantics

Context is retrieved independently from three sources.



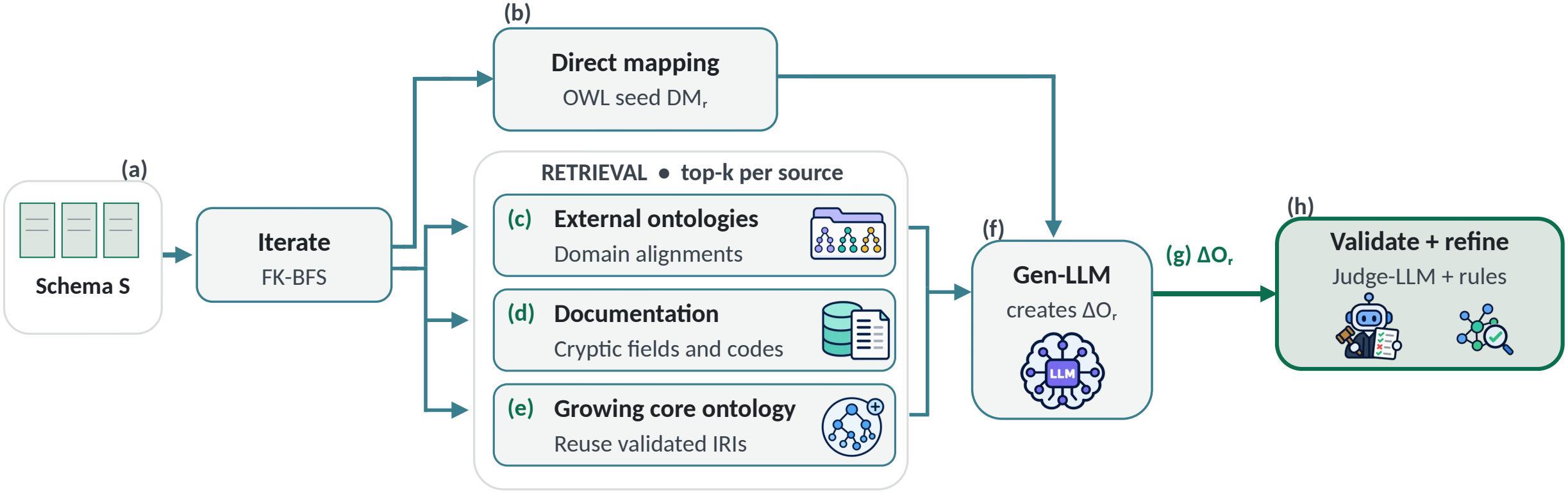
4. Generate a delta ontology

The Gen-LLM receives the direct mapping and the retrieved context.



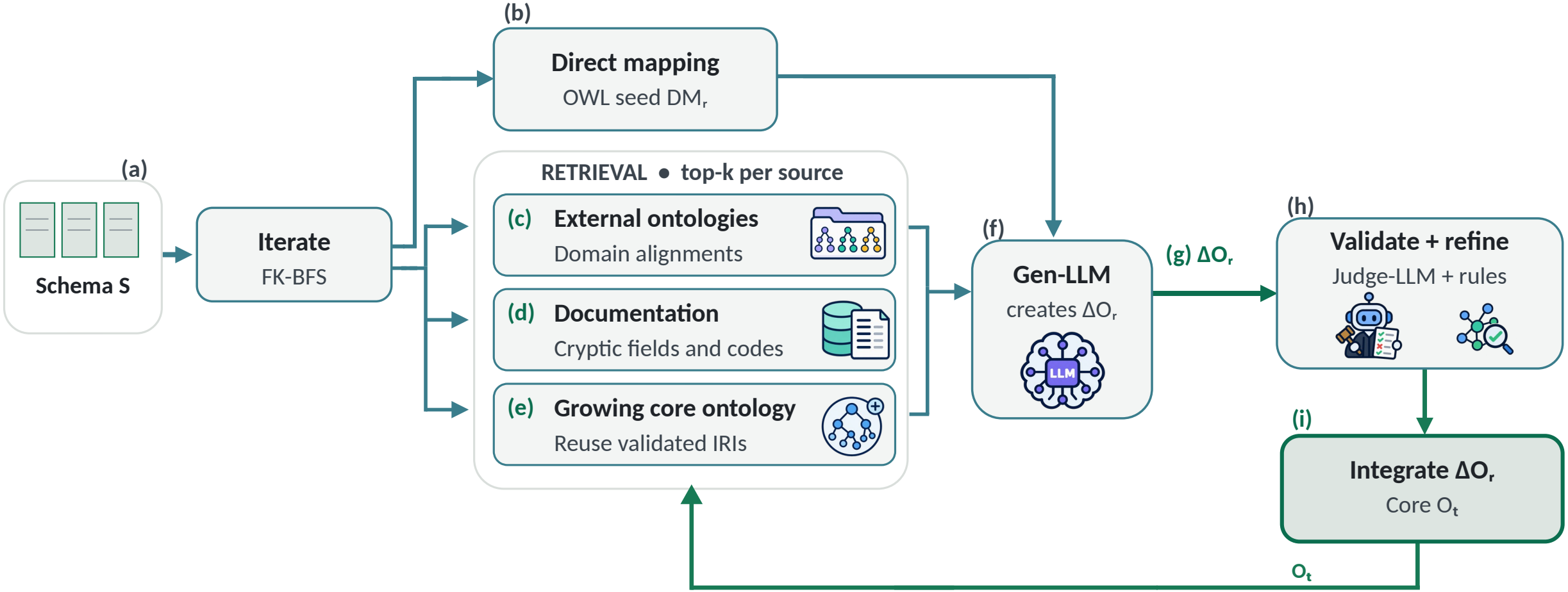
5. Validate before integration

Validate and refine each delta before it joins the core.



6. Integrate and reuse the growing core

The growing core feeds the next generation iteration.



EVALUATION

Does the method work beyond our running example?

Three databases, **three** generator models, **four** methods, evaluated from complementary **angles**.

LIVER CANCER



16 tables · 350 columns

specialist clinical registry

Universitätsklinikum Leipzig

eICU-CRD



31 tables · 559 columns

large ICU schema

PhysioNet [Pollard et al., Sci. Data 2018]

Medical Domain

EVALUATION

Does the method work beyond our running example?

Three databases, **three** generator models, **four** methods, evaluated from complementary **angles**.

LIVER CANCER



16 tables · 350 columns

specialist clinical registry

Universitätsklinikum Leipzig

eICU-CRD



31 tables · 559 columns

large ICU schema

PhysioNet [Pollard et al., Sci. Data 2018]

Medical Domain

CHINOOK



11 tables · 67 columns

music store

Chinook Database [Rocha n.d.; accessed 2026]

Music Domain

EVALUATION

Does the method work beyond our running example?

Three databases, **three** generator models, **four** methods, evaluated from complementary **angles**.

LIVER CANCER



16 tables · 350 columns

specialist clinical registry

Universitätsklinikum Leipzig

eICU-CRD



31 tables · 559 columns

large ICU schema

PhysioNet [Pollard et al., Sci. Data 2018]

CHINOOK



11 tables · 67 columns

music store

Chinook Database [Rocha n.d.; accessed 2026]

FOUR METHODS TO GENERATE AN ONTOLOGY FROM A RELATIONAL DATABASE SCHEMA

1 Direct Mapping

Deterministic

Direct Mapping [Sequeda et al., WWW 2012]

2 LLM baseline

Table only

Ontology engineering with large language models [Mateiu & Groza, SYNASC 2023]

3 Non-Iterative RAG

One-pass RAG

4 RIGOR

Ground + grow + check

LLM-based Methods

EVALUATION

Does the method work beyond our running example?

Three databases, **three** generator models, **four** methods, evaluated from complementary **angles**.

LIVER CANCER



16 tables · 350 columns

specialist clinical registry

Universitätsklinikum Leipzig

eICU-CRD



31 tables · 559 columns

large ICU schema

PhysioNet [Pollard et al., Sci. Data 2018]

CHINOOK



11 tables · 67 columns

music store

Chinook Database [Rocha n.d.; accessed 2026]

FOUR METHODS TO GENERATE AN ONTOLOGY FROM A RELATIONAL DATABASE SCHEMA

1 Direct Mapping

Deterministic

Direct Mapping [Sequeda et al., WWW 2012]

2 LLM baseline

Table only

Ontology engineering with large language models [Mateiu & Groza, SYNASC 2023]

3 Non-Iterative RAG

One-pass RAG

4 RIGOR

Ground + grow + check

LLM-based Methods

3 **LLM METHODS** × 3 **DATABASES** × 3 **LLMS** = **27 ONTOLOGIES**

EVALUATION

Does the method work beyond our running example?

Three databases, **three** generator models, **four** methods, evaluated from complementary **angles**.

LIVER CANCER



16 tables · 350 columns

specialist clinical registry

Universitätsklinikum Leipzig

eICU-CRD



31 tables · 559 columns

large ICU schema

PhysioNet [Pollard et al., Sci. Data 2018]

CHINOOK



11 tables · 67 columns

music store

Chinook Database [Rocha n.d.; accessed 2026]

FOUR METHODS TO GENERATE AN ONTOLOGY FROM A RELATIONAL DATABASE SCHEMA

1 Direct Mapping

Deterministic

Direct Mapping [Sequeda et al., WWW 2012]

2 LLM baseline

Table only

Ontology engineering with large language models [Mateiu & Groza, SYNASC 2023]

3 Non-Iterative RAG

One-pass RAG

4 RIGOR

Ground + grow + check

LLM-based Methods

1 **METHOD** × 3 **DATABASES** = 3 ONTOLOGIES

3 **LLM METHODS** × 3 **DATABASES** × 3 **LLMS** = 27 ONTOLOGIES

EVALUATION

Does the method work beyond our running example?

Three databases, **three** generator models, **four** methods, evaluated from complementary **angles**.

LIVER CANCER



16 tables · 350 columns

specialist clinical registry

Universitätsklinikum Leipzig

eICU-CRD



31 tables · 559 columns

large ICU schema

PhysioNet [Pollard et al., Sci. Data 2018]

CHINOOK



11 tables · 67 columns

music store

Chinook Database [Rocha n.d.; accessed 2026]

FOUR METHODS TO GENERATE AN ONTOLOGY FROM A RELATIONAL DATABASE SCHEMA

1 Direct Mapping

Deterministic

Direct Mapping [Sequeda et al., WWW 2012]

2 LLM baseline

Table only

Ontology engineering with large language models [Mateiu & Groza, SYNASC 2023]

3 Non-Iterative RAG

One-pass RAG

4 RIGOR

Ground + grow + check

LLM-based Methods

1 **METHOD** × 3 **DATABASES** = 3 ONTOLOGIES

3 **LLM METHODS** × 3 **DATABASES** × 3 **LLMS** = 27 ONTOLOGIES

3+27 = 30 ONTOLOGIES

EVALUATION

Does the method work beyond our running example?

Three databases, **three** generator models, **four** methods, evaluated from complementary **angles**.

LIVER CANCER



16 tables · 350 columns

specialist clinical registry

Universitätsklinikum Leipzig

eICU-CRD



31 tables · 559 columns

large ICU schema

PhysioNet [Pollard et al., Sci. Data 2018]

CHINOOK



11 tables · 67 columns

music store

Chinook Database [Rocha n.d.; accessed 2026]

FOUR METHODS TO GENERATE AN ONTOLOGY FROM A RELATIONAL DATABASE SCHEMA

1 Direct Mapping

Deterministic

Direct Mapping [Sequeda et al., WWW 2012]

2 LLM baseline

Table only

Ontology engineering with large language models [Mateiu & Groza, SYNASC 2023]

3 Non-Iterative RAG

One-pass RAG

4 RIGOR

Ground + grow + check

30 ONTOLOGIES → checked by quality checks · LLM-as-a-Judge (GPT-5.4) · 10 human experts · BURR benchmark [1]

[1] BURR benchmark [Laskowski et al., PACMMOD 2025]

RESULT 1 / VALID, WITH FEWER ERRORS

Every ontology is valid. RIGOR's have far fewer modeling errors.

All 30 ontologies **parse without errors** (RDFLib) and are **logically consistent** (HermiT). So we look deeper.

1 VALID, FEWER ERRORS | 2 KEEPS THE SCHEMA | 3 ADDS MEANING | 4 GOLD MAPPINGS | 5 JUDGES

MODELING ERRORS FOUND BY OOPS!

pitfalls per ontology, fewer is better

	Liver Cancer			eICU-CRD			Chinook			Avg
Direct Mapping	731			665			102			499
	Claude	Mistral	DeepSeek	Claude	Mistral	DeepSeek	Claude	Mistral	DeepSeek	
LLM baseline	422	397	403	434	440	421	95	74	81	307
Non-Iterative RAG	29	61	18	39	220	27	24	35	14	52
RIGOR	**20**	**23**	**3**	**2**	**2**	**26**	**0**	**0**	**0**	**8**

Same LLM, same database: **Mistral on eICU, 220 with Non-Iterative RAG**, **2 with RIGOR.** On Chinook, RIGOR has zero with every LLM.

OOPS! [Poveda-Villalón et al., IJSWIS 2014]

RESULT 2 / NOTHING IS LOST

Nothing is lost: RIGOR keeps the schema.

1 VALID, FEWER ERRORS | 2 KEEPS THE SCHEMA | 3 ADDS MEANING | 4 GOLD MAPPINGS | 5 JUDGES

KEEPS THE SCHEMA

columns covered, per ontology (%)

	Liver Cancer			eICU-CRD			Chinook			Avg
	Claude	Mistral	DeepSeek	Claude	Mistral	DeepSeek	Claude	Mistral	DeepSeek	
Direct Mapping	97			99			78			91%
LLM baseline	94	31	89	96	68	86	100	50	52	74%
Non-Iterative RAG	41	6	12	22	28	46	58	16	78	34%
RIGOR	**91**	**99**	**87**	**100**	**99**	**77**	**95**	**95**	**95**	**93%**

Coverage: a column counts as covered when an ontology class or property name is similar to it (embedding similarity ≥ 0.55).

RESULT 2 / NOTHING IS LOST

Nothing is lost: RIGOR keeps the schema.

1 VALID, FEWER ERRORS | **2 KEEPS THE SCHEMA** | 3 ADDS MEANING | 4 GOLD MAPPINGS | 5 JUDGES

KEEPS THE SCHEMA

columns covered, per ontology (%)

	Liver Cancer			eICU-CRD			Chinook			Avg
	Claude	Mistral	DeepSeek	Claude	Mistral	DeepSeek	Claude	Mistral	DeepSeek	
Direct Mapping	97			99			78			91%
LLM baseline	94	31	89	96	68	86	100	50	52	74%
Non-Iterative RAG	41	6	12	22	28	46	58	16	78	34%
RIGOR	**91**	**99**	**87**	**100**	**99**	**77**	**95**	**95**	**95**	**93%**

Non-Iterative RAG 6–78%. With RIGOR: **≥ 77% for every LLM, in every database.**

Coverage: a column counts as covered when an ontology class or property name is similar to it (embedding similarity ≥ 0.55).

RESULT 3 / MEANING IS ADDED

Meaning is added: labels, provenance and hierarchy.

1 VALID, FEWER ERRORS | 2 KEEPS THE SCHEMA | **3 ADDS MEANING** | 4 GOLD MAPPINGS | 5 JUDGES

ADDS MEANING

		Liver Cancer			eICU-CRD			Chinook			Avg
		Claude	Mistral	DeepSeek	Claude	Mistral	DeepSeek	Claude	Mistral	DeepSeek	
	Direct Mapping LLM baseline	*No labels, provenance or class hierarchy, in any run.*									
Labels & comments	Non-Iterative RAG	680	134	736	877	157	743	98	14	64	389
	RIGOR	**1,029**	**1,137**	**341**	**1,291**	**1,684**	**435**	**357**	**362**	**359**	**777**
Provenance links	Non-Iterative RAG	0	0	0	0	0	0	0	0	0	0
	RIGOR	**336**	**308**	**22**	**423**	**335**	**99**	**81**	**83**	**75**	**196**
Class hierarchy *(inheritance richness)*	Non-Iterative RAG	0	0	0	0	0	0	0	0	0	0
	RIGOR	**0.70**	**0.39**	**0.13**	**0.62**	**0.49**	**0.10**	**2.12**	**2.72**	**4.07**	**1.26**

Only RIGOR adds **labels**, **provenance and** a **class hierarchy**: **with every LLM, in every database.**

Labels & comments: annotation assertions; provenance: prov:wasDerivedFrom.
Hierarchy: inheritance richness, subclass axioms per class [**OntoQA:** Tartir et al., KADASH 2005; **OntoMetrics:** Lantow, KEOD 2016].

RESULT 4 / EXTERNAL BENCHMARK

RIGOR keeps the link to the data.

BURR [Laskowski et al., PACMMOD 2025]

1 VALID, FEWER ERRORS | 2 KEEPS THE SCHEMA | 3 ADDS MEANING | **4 GOLD MAPPINGS** | 5 JUDGES

WHAT BURR CHECKS

Every ontology element must map back to the database.

CLASSES	`person`	→	`Class: Person`
RELATIONS	`paper.author_id`	→	`ObjectProperty: author`
ATTRIBUTES	`person.name`	→	`DataProperty: name`

SYSTEM	CLASSES	RELATIONS	ATTRIBUTES
W3C-Mapper / Ontop rule-based	0.53	0.11	0.39
RDB2Onto rule-based	*0.67*	*0.62*	*0.44*
OntoGenix LLM-based	0.58	0.35	*0.44*
RIGOR LLM + rules	**0.73**	**0.67**	**0.52**

RESULT 5 / LLM JUDGE AND HUMAN EXPERTS

An LLM judge and human experts both rank RIGOR first.

Blind evaluation: nobody knew which ontology came from which method.

1 VALID, FEWER ERRORS | 2 KEEPS THE SCHEMA | 3 ADDS MEANING | 4 GOLD MAPPINGS | **5 JUDGES**

HOW THEY JUDGED For each table: its **schema**, **5 competency questions** and **four anonymized ontologies**. Each **scored 0–5** on ***accuracy***, ***completeness***, ***conciseness***, ***adaptability***, ***clarity***, ***consistency*** and ***domain enrichment***, then ranked **1st–4th**.

LLM-AS-A-JUDGE · GPT-5.4

100% of rankings put RIGOR 1st

	Liver Cancer	eICU-CRD
Direct Mapping	2.05	2.13
LLM baseline	1.38	0.28
Non-Iterative RAG	1.89	0.67
RIGOR	**3.93**	**3.64**

10 ONTOLOGY EXPERTS

70% of rankings put RIGOR 1st

	Liver Cancer	eICU-CRD
Direct Mapping	2.95	2.59
LLM baseline	2.74	3.01
Non-Iterative RAG	2.39	1.24
RIGOR	**3.44**	**3.53**

Quality dimensions: accuracy, completeness, conciseness, adaptability, clarity, consistency [Vrandečić, Handbook on Ontologies 2009]; domain enrichment, adapted from ontology enrichment [Petasis et al., LNCS 2011].

FUTURE WORK

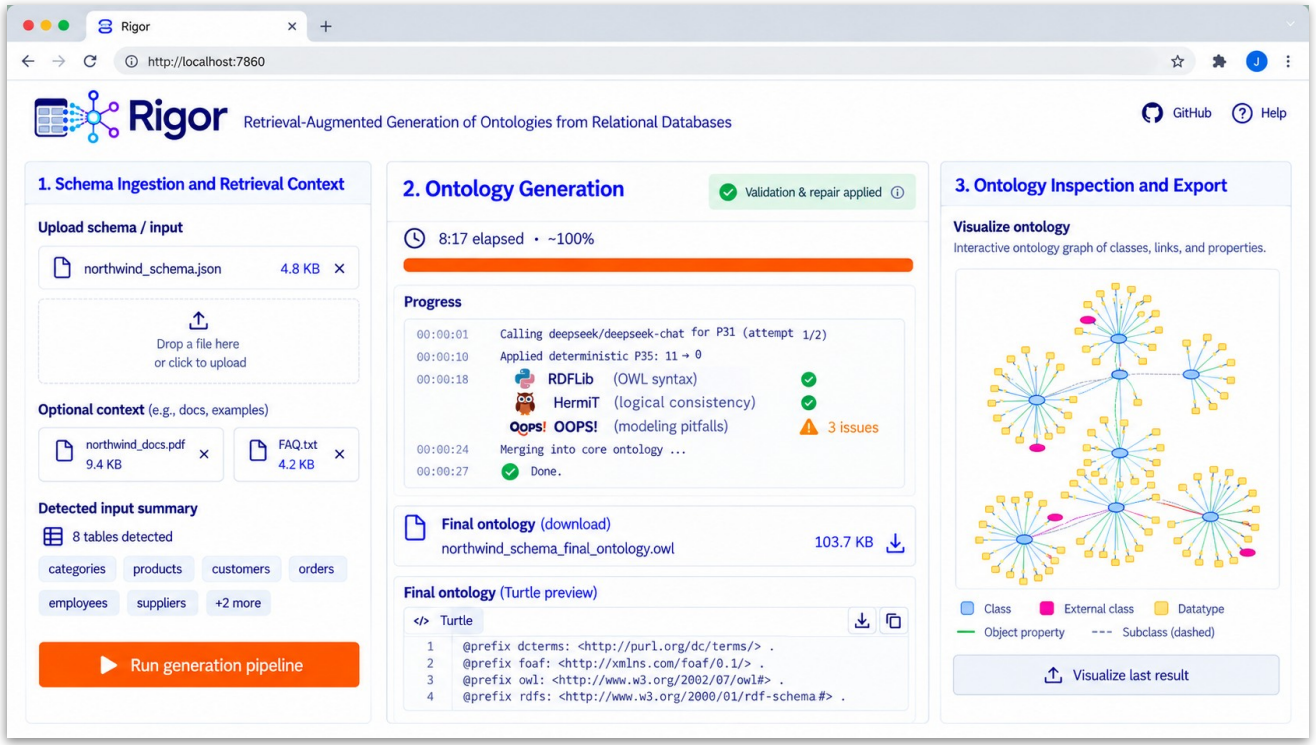
Next: Move the evaluation tools into the loop

RDFLib, the **reasoner** and **OOPS!** only checked our final ontologies. Next, they flag each error while ***RIGOR*** builds, and their **feedback** will be used to **prompt the LLM** to **repair** the ontology.

FUTURE WORK

Next: Move the evaluation tools in the loop

RDFLib, the **reasoner** and **OOPS!** only checked our final ontologies. Next, they flag each error while ***RIGOR*** builds, and their **feedback** will be used to **prompt the LLM** to **repair** the ontology. → *Live in our RIGOR App demo.*



RIGOR App [Fathallah, Nayyeri & Staab, ISWC 2026 Demos].

Open source · MIT

Watch the demo video

FUTURE WORK

Next: let the data speak.

Today ***RIGOR*** reads **schemas**. **Instance data** can propose **new knowledge**.

1 THE DATA

two tables, joined on admission_id

treatment

admission_id FK	drug
A1	cisplatin
A2	nivolumab
A3	cisplatin

admission

admission_id	patient_id	BM_DEP
A1	P1	1
A2	P2	0
A3	P3	1

FUTURE WORK

Next: let the data speak.

Today ***RIGOR*** reads **schemas**. **Instance data** can propose **new knowledge**.

1 THE DATA

two tables, joined on admission_id

treatment

admission_id FK	drug
A1	cisplatin
A2	nivolumab
A3	cisplatin

admission

admission_id	patient_id	BM_DEP
A1	P1	1
A2	P2	0
A3	P3	1

2 THE PATTERN

what the joined rows show

Every cisplatin admission has bone marrow depression.



admissions with BM_DEP = 1

3 CANDIDATE KNOWLEDGE

A reviewable hypothesis

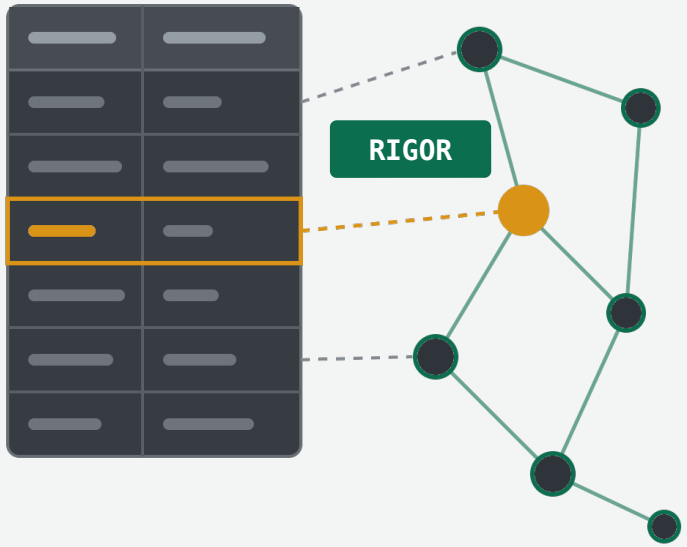
```
Individual: cisplatin_bmdep
  Types: CandidateAssociation
  Facts:
    hasDrug Cisplatin,
    hasCondition BoneMarrowDepression,
    status "needs expert review"
  Annotations:
    prov:wasDerivedFrom treatment:drug,
    prov:wasDerivedFrom admission:BM_DEP
```

A PATTERN → an expert reviews it before it joins the ontology

Thank you.

Questions?

FROM A DATABASE SCHEMA TO AN ONTOLOGY



PAPER + OPEN SOURCE

Paper

GitHub

📬 nadeen.fathallah@ki.uni-stuttgart.de